

Scallop VeSCA

Audit Report

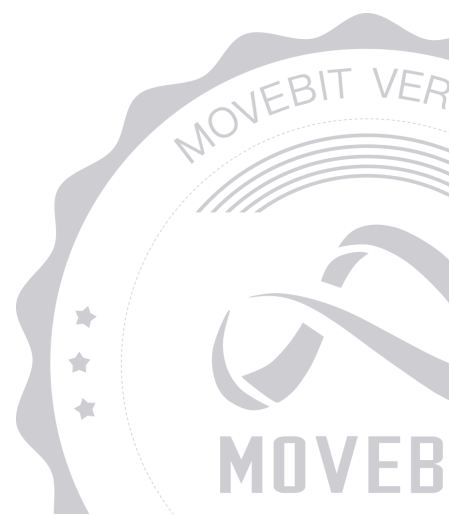


contact@bitslab.xyz



https://twitter.com/movebit_

Fri Jun 20 2025



Scallop VeSCA Audit Report

1 Executive Summary

1.1 Project Information

Description	Scallop VeSCA is a staking vesting protocol
Type	DeFi
Auditors	MoveBit
Timeline	Wed Jun 04 2025 - Fri Jun 20 2025
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/scallop-io/ve-sca
Commits	a6bc3f02359cb193a08c0a923f7cb5c95370cc9137e01d0ace7472447bf2314f8602f5c5dffa962f

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
MPR	Move.production.toml	c889bb2950bb3683519909ff164a62b805fb20b7
VSUS	sources/ve_sca_unlock_schedule.move	7b52a0c39ef3c692776ef82c4a24c6a231cdcb05
UTI	sources/util.move	3feb5749e653ba2bff92f7bf8e8969adf719629c
VSR	sources/ve_sca_rules.move	3ae938095a5d199caf3c76d88c219479e25693ae
CON	sources/constants.move	02f37ee21457f073accd609b35c62ffa6b20f239
ECO	sources/error_code.move	4817a7d60b3994bb77790bd889d551e803bdad40
VST	sources/ve_sca_treasury.move	d6e1c749dc2b5c1ffd8708f77944fa d15bcc6334
VSC	sources/ve_sca_calculator.move	ea6f1c2ebe7d795088086924d68164b6beea88a1
VSD	sources/ve_sca_display.move	ac2cb87c80dc3f4ec6949d1bcfe255d245b3587a
VSC1	sources/ve_sca.move	dff41144492dc844ddd2852001ad4d4cc303af01

VSS	sources/ve_sca_subscriber.move	7c2473b1671d347f728ac51daa0bdc06677d9328
CON1	sources/config.move	dae55b44dd36d1f05b76d7708f99a860dacbf8e5

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	2	2	0
Informational	1	1	0
Minor	1	1	0
Medium	0	0	0
Major	0	0	0
Critical	0	0	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the "**Testing and Automated Analysis**", "**Code Review**" and "**Formal Verification**" strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by [Scallop](#) to identify any potential issues and vulnerabilities in the source code of the [Scallop VeSCA](#) smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 2 issues of varying severity, listed below.

ID	Title	Severity	Status
CON-1	Redundant Initialization Operation	Minor	Fixed
VSS-1	Condition Check Optimization	Informational	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the [Scallop VeSCA](#) Smart Contract :

Admin

- `upgrade_protocol_version` : Upgrade protocol version
- `disable_protocol` : Disable protocol
- `enable_protocol` : Enable protocol
- `whitelist_subscriber` : Whitelist a subscriber for veSCA
- `de_whitelist_subscriber` : De-whitelist a subscriber for veSCA
- `extend_lock_period_by_admin` : Extend the lock period by admin

User

- `subscribe` : Subscribe to a veSCA key
- `unsubscribe` : Unsubscribe from a veSCA key
- `mint_ve_sca_key` : Mint a veSCA key with locked SCA
- `lock_more_sca` : Lock additional SCA into a veSCA
- `extend_lock_period` : Extend veSCA unlock time
- `renew_expired_ve_sca` : Renew an expired veSCA
- `redeem` : Redeem locked SCA from veSCA
- `mint_ve_sca_placeholder_key` : Mint placeholder veSCA key with 0 lock
- `split` : Split a veSCA key into two
- `merge` : Merge two veSCA keys into one

4 Findings

CON-1 Redundant Initialization Operation

Severity: Minor

Status: Fixed

Code Location:

sources/config.move#80-85

Descriptions:

In the `config` contract, the `init` function initializes the `ve_sca_subscriber` by calling `ve_sca_subscriber::initialize(ctx)`. However, there is also an additional function named `initialize_subscribers` that performs the same initialization. This could lead to redundant or repeated initialization.

```
fun init(ctx: &mut TxContext) {
  let config = VeScaProtocolConfig {
    id: object::new(ctx),
    version: CURRENT_VE_SCA_PROTOCOL_VERSION,
    enabled: true,
  };
  let admin_cap = VeScaProtocolAdminCap {
    id: object::new(ctx),
  };
  ve_sca_subscriber::initialize(ctx);
  transfer::share_object(config);
  transfer::transfer(admin_cap, tx_context::sender(ctx));
}

public fun initialize_subscribers(
  _admin_cap: &VeScaProtocolAdminCap,
  ctx: &mut TxContext,
){
```

```
ve_sca_subscriber::initialize(ctx);  
}
```

Suggestion:

Remove the redundant initialization to avoid potential repeated execution and maintain consistent contract behavior.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

VSS-1 Condition Check Optimization

Severity: Informational

Status: Fixed

Code Location:

sources/ve_sca_subscriber.move#53-56

Descriptions:

In the `subscribe` function, the following condition is used:

```
if (vec_set::contains(subscribers, &subscriber_name) == false) {  
    vec_set::insert(subscribers, subscriber_name);  
}
```

This can be simplified using the `!` operator for better readability and consistency with the `unsubscribe` function, which uses:

```
if (vec_set::contains(subscribers, &subscriber_name)) {  
    vec_set::remove(subscribers, &subscriber_name);  
}
```

Suggestion:

Update the `subscribe` function condition as follows to align with the `unsubscribe` style and improve code clarity:

```
if (!vec_set::contains(subscribers, &subscriber_name)) {  
    vec_set::insert(subscribers, subscriber_name);  
}
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

